

Suffix Array

Kanishk Goel

7 May, 2026

1 Suffixes

For a string abcde, suffixes are abcde, bcde, cde, de and e.

2 Suffix Array

A suffix array contains empty string as well. It is the lexicographical order of all suffixes. Storing it would require $O(n^2)$ space, so we store the indexes of the strings at the place which they start in the original string.

For example - ababba

0. (6)
1. a (5)
2. ababba (0)
3. abba (2)
4. ba (4)
5. babba (1)
6. bba (3)

where $(\cdot)$ indicates the numbering of the index i.e the way it is stored in the suffix array.

The algorithm is made simpler by adding a new symbol which is treated as the smallest symbol in place of the empty string, then everything is made of in powers of 2 by adding the cyclic string as filler strings in the suffixes. Each filler character is therefore, simply its *index mod (length of string)*.

3 Algorithm Idea

Start with base $k = 0$, this means we are comparing all strings by the first character. Use any $O(n \log n)$ sort for this step. The algorithm is divided into $\log n$ iterations. For k^{th} iteration, the string s contains $s[i \dots i + 2^k - 1]$ slice. (Here i is the index of the suffix and is computed $i \bmod n$)

To compare string of length 2^{k+1} , we first compare the number we gave to left part of the string, if lefts are unequal, we don't need to compare the right, else compare right.

Done quickly through use of **Prefix Doubling**.

4 Prefix Doubling

DP technique to compare 2^n length strings.

4.1 Base Case(2^0)

You compare the length 1 strings directly and assign equivalence classes. For example a,b,a get 1,2,1.

4.2 Transition 2^k to 2^{k+1}

Divide into 2^{k+1} strings and for k the classes have been computed. Compare left half, if equal compare right half. So, the comparison step becomes $O(1)$.

4.3 The DP

For example, banana is the string, for $k = 0$, we insert into $DP[k][i]$ (null strings are represented as \$) as follows:

$$DP[k=0] = [b,a,n,a,n,a,\$] = [2,1,3,1,3,1,0]$$

$$DP[k=1] = [ba,an,na,an,na,a\$, \$b] = [3,2,4,2,4,1,0]$$

according to the rule of pairs ($DP[k=1][i], DP[k=1][i + 2^{k-1}]$), therefore, what was sorted - [(2,1),(1,3) and so on]

5 Substring Search

Question: Suppose we have a string s and need to answer requests of string p , i.e whether it is a substring of string s or not.

Solution: Cut off $|p|$ part of each substring, we get a suffix array, with sorted $|p|$ length strings, use binary search, to find the string p now.

Runtime: $O(|p| \log n)$

6 LCP Array

Problem: Largest Common Prefix Between Two Suffixes
Input: Pair (i, j), the pair of suffix substrings in the suffix array to find the largest common prefix in. Output: An LCP array that allows for finding largest common prefix between any pairs of suffixes in the suffix array in linear time.

6.1 Idea

Build an lcp array, with largest common prefixes of consecutive strings, in the order of suffix array. Then for any pair, (i, j) , traverse this lcp array from i to j (assuming $i < j$), and the answer is $\min(lcp[k]) \forall k \in [i, j]$.

Calculating the minimum on a segment of lcp array is done through sparse table ($O(n \log n)$ construction and $O(1)$ retrieval) or segment tree ($O(n)$ construction and $O(\log n)$ retrieval).

6.2 How to Build

Let suffix j be before i in the suffix array and they have $k > 0$ common characters. Then the suffixes $i + 1$ and $j + 1$ have $k - 1$ common characters at the very least. So, we start from $k - 1$.

Time Complexity: $O(n \log n)$ to build the suffix array, and $O(n)$ for lcp array building.

7 Applications

7.1 Number of Different Substrings

Problem: Number of Different Substrings
Input: String s. Output: Number of different substrings of s.

Solution: Let length of string be n . Number of repeated substrings equals $\frac{n(n+1)}{2}$. Number of substrings that have been repeated are included in the LCP

array. This is because each substring is a prefix of a suffix in the suffix array. Hence, the answer is simply $\frac{n(n+1)}{2}$ – sum of lcp array.

7.2 Longest Common Substring

Problem: Longest Common Substring
Input: String s and t . Output: Longest string common to both t and s .

Solution: Combine s and t using a delimiter not found in both strings. Divide the suffix array thus formed into two classes, those from s and those from t . Now find the largest $LCP(i, j)$ for consecutive i and j such that one belongs from class s and other from class t .